WEEK 8 PRACTICAL

Task1: Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), realloc(), and free(). Also identify and analyze memory leaks using Valgrind.Aim

Program

```
  GNU nano 7.2                    dynamic_memory.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;

    // malloc
    p = malloc(2 * sizeof(int));
    p[0] = 10;
    p[1] = 20;

    printf("After malloc: %d %d\n", p[0], p[1]);

    // realloc
    p = realloc(p, 3 * sizeof(int));
    p[2] = 30;

    printf("After realloc: %d %d %d\n", p[0], p[1], p[2]);

    // free
    free(p);

    // calloc
    p = calloc(2, sizeof(int));

    printf("After calloc: %d %d\n", p[0], p[1]);

    // Intentionally not freeing p
    // This creates a memory leak

    return 0;
}
```

Output

```
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ pwd
/home/suraj/OSSP/Week8
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ nano dynamic_memory.c
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ gcc dynamic_memory.c -o dynamic_memory
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ ./dynamic_memory
After malloc: 10 20
After realloc: 10 20 30
After calloc: 0 0 0
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ valgrind --leak-check=full ./dynamic_memory
==948== Memcheck, a memory error detector
==948== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==948== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==948== Command: ./dynamic_memory
After malloc: 10 20
After realloc: 10 20 30
After calloc: 0 0 0
==948==
==948== HEAP SUMMARY:
==948==     in use at exit: 8 bytes in 1 blocks
==948==   total heap usage: 4 allocs, 3 frees, 1,052 bytes allocated
==948==
==948== 8 bytes in 1 blocks are definitely lost in loss record 1 of 1
==948==    at 0x484D953: calloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==948==    by 0x10928E: main (in /home/suraj/OSSP/Week8/dynamic_memory)
==948==
==948== LEAK SUMMARY:
==948==    definitely lost: 8 bytes in 1 blocks
==948==    indirectly lost: 0 bytes in 0 blocks
==948==      possibly lost: 0 bytes in 0 blocks
==948==    still reachable: 0 bytes in 0 blocks
==948==         suppressed: 0 bytes in 0 blocks
==948== For lists of detected and suppressed errors, rerun with: -s
==948== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Observation

The program successfully demonstrated dynamic memory allocation using malloc(), calloc(), realloc(), and free(). malloc() allocated memory for integers, realloc() increased the allocated memory size, and calloc() allocated memory initialized to zero. Valgrind detected an 8-byte memory leak because the memory allocated using calloc() was not freed. After using free() properly, the allocated memory was released successfully.

Program

```
 GNU nano 7.2                    copy_on_write.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int data = 10;

    printf("Before fork():\n");
    printf("Parent data: %d\n", data);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("\nChild process:\n");
        printf("Before modification: %d\n", data);

        data = 99;

        printf("After modification: %d\n", data);
    }
    else
    {
        wait(NULL);

        printf("\nParent process:\n");
        printf("Parent data: %d\n", data);
    }

    return 0;
}
```

Output

```
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ cd ~/OSSP/Week8
saraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ nano copy_on_write.c
saraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ gcc copy_on_write.c -o copy_on_write
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$ ./copy_on_write

Before fork():
Parent data: 10

Child process:
Before modification: 10
After modification: 99

Parent process:
Parent data: 10
suraj@DESKTOP-BLT5L3R:~/OSSP/Week8$
```

Observation

The program demonstrated the Copy-on-Write mechanism using fork(). Initially, both the parent and child processes had the same value of the variable. When the child modified the variable, its value changed only in the child process, while the parent retained the original value. This shows that the operating system provides separate copies of the data when a modification occurs.